

Airport Management System

Project Proposal for UCS503P
Submitted to: Dr. Raghav B. Venkataramaiyer
Thapar Institute of Engineering and Technology

TODO Name, CSED*

September 9, 2026

1 Higher-order goal

... secondary framing

This project aims to demonstrate that a single, well-designed operational data layer – shared across every subsystem of an airport – can support a wide range of day-to-day airport functions (flight operations, passenger-facing services, staff management, security, and facilities) without each subsystem owning a private data silo. The immediate engineering objective is to translate this architectural claim into a deployable, multi-module web platform.

2 Time-to-value

... primary heuristic

- We will deliver a live, working skeleton (database + API + frontend health check) before any feature work begins, so every subsequent module ships against a real, deployed system.
- Modules are explicitly tiered: **Tier 1** modules (concurrency, cascading updates, real-time state, prediction) are built first for engineering depth; **Tier 2** modules (CRUD + status pipelines) follow once the core patterns are proven, so complexity is front-loaded by design rather than discovered under deadline pressure.
- Success in each weekly iteration is defined by what is deployed and demonstrable that week, not by a big-bang release at the end.

3 Problem Statement

Airports run many operationally distinct but data-coupled functions – flight scheduling, gate assignment, baggage handling, passenger booking, staff rostering, security screening, parking, and facilities – that in practice are often built as separate systems stitched together after the fact. This causes:

- **Data duplication and drift:** the same flight, gate, or passenger record is re-entered or re-synced across systems, creating inconsistency.
- **Delayed cascades:** a single flight delay should ripple through gate reassignment, connecting-flight risk, staff scheduling, and passenger notifications – but siloed systems propagate this slowly or not at all.

*Roll No: 10xxxxxxx, Email: TOD0thapar.edu

- **Concurrency hazards:** shift scheduling, resource assignment, and parking-slot booking are all instances of the same conflict-detection problem, usually re-solved (and re-bugged) independently in each silo.
- **Fragmented staff tooling:** ground crew, security, medical, and admin staff each need role-specific views into the same underlying operational state.

Why this matters now (contextual relevance): even a teaching-scale airport platform surfaces the same architectural pressure real systems face – one shared operational core, read and written by many role-specific modules, is what actually prevents the inconsistency and cascade-delay problems above.

4 Proposed Solution

... Software Proposition

4.1 Overview

Build a **web-based** Airport Management System around one shared operational core (**Airport, Gate, Flight, Passenger, Resource, User**), with the following module groups on top of it:

- **Ops Core:** flights, gates, resources, turnaround, and delay handling with cascading recalculation.
- **Passenger-facing:** booking portal, check-in, connecting-flight risk, baggage tracking, accessibility & medical assistance requests, a support chatbot, parking, and a facilities directory.
- **Operations & safety:** emergency handling, security checkpoints/screening/incidents, predictive analytics for delay and congestion.
- **Staff:** Staff & Employee Management – records, shift scheduling with conflict prevention, and attendance.
- **Admin Dashboard:** rolls up all of the above.

4.2 Core workflow

1. Ops staff maintain flights, gates, and resources against the shared core; a delay on any flight triggers cascading updates to dependent gate/resource/connecting-flight state.
2. Passengers interact with the same core through booking, check-in, baggage status, and the chatbot – all reading live operational data rather than a stale copy.
3. Staff are scheduled onto shifts through a single conflict-checking service; the same pattern is reused for resource assignment and parking-slot booking.
4. The Admin Dashboard aggregates status and metrics across every module from the shared core.

4.3 Operational constraints for an educational setting

- Role-based access (passenger, ground_crew, security, medical, ops_manager, admin) enforced uniformly via JWT middleware rather than per-module ad hoc checks.
- Avoid dependence on external/paid services where a mock or rule-based substitute suffices (e.g. mock payments, rule-based chatbot over live DB data) – focus stays on workflow and system correctness.
- Design for deployability using common web hosting and a managed Postgres instance.

5 Solution Approach

... Engineering focus

This is an engineering course project, so the focus is on the shared data layer, correctness of concurrent operations, and workflow – not on research-grade algorithms.

5.1 Conflict detection

... reused pattern, not novel research

Shift scheduling, resource assignment, and parking-slot booking all reduce to the same overlap-detection problem over time ranges:

- A single, unit-tested conflict-checking service (`shiftConflict.js`) is written once against shifts and reused as-is for resource assignment and parking.
- On conflict, the API returns 409 with the conflicting rows so the caller (staff UI) can resolve it, rather than silently overwriting or rejecting without explanation.

5.2 Web architecture

... web-based solution

A standard 3-tier web architecture:

- **Frontend:** React (Vite), role-aware views for passengers, staff, and admins.
- **Backend API:** Node.js + Express, JWT auth, Socket.IO for real-time updates (delays, gate changes).
- **Database:** PostgreSQL, one schema covering all 15 modules' entities against the shared operational core.

5.3 Testing and delivery

- Jest (backend) for unit tests on pure logic (e.g. conflict detection) that don't require a live database; React Testing Library planned for frontend component tests.
- Each week's module ships against the already-deployed skeleton, so the live system is always in a demonstrable state.

6 Evaluation Criterion

... measurable and attributable

6.1 Primary evaluation metric

Correctness of shared-core consistency: whether a state change in one module (e.g. a flight delay) is correctly and promptly reflected in every dependent module (gate status, connecting-flight risk, passenger notification) without manual re-entry.

- Target: cascading updates visible across dependent modules within the same request/socket cycle that recorded the delay.
- Attribution: verified via integration tests and manual cross-module checks during weekly demos.

6.2 Secondary evaluation metrics

- **Conflict-detection correctness:** unit-test coverage of the shared overlap-checking service across shifts, resource assignment, and parking.
- **API health:** `/health` reports `db.connected`: `true` in every deployed environment.

- **Role isolation:** each protected route enforces the correct role set, verified by auth middleware tests.
- **Weekly deployability:** each week's increment builds and deploys without breaking the previous week's live system.

7 Scalability

...with foresight

1. **Scalable (theoretically):** decoupled frontend/backend, stateless JWT-authenticated API, indexed queries on the shared core tables, and a conflict-checking service designed to be reused (not re-implemented) by every module that needs it.
2. **Operationally deployable (practically):** managed Postgres, platform-hosted backend (Render/Railway) and frontend (Vercel), configuration entirely via environment variables.

8 Engine availability heuristic

- Web framework (Express) for REST APIs and Socket.IO for real-time push.
- Managed relational database (Postgres via Render/Neon/Supabase).
- Token-based authentication (JWT) with role middleware.
- Test framework (Jest / React Testing Library).
- Platform-hosted CD targets (Render/Railway, Vercel) instead of self-managed infrastructure.

These mature components let the project focus on the shared data model, cascading/conflict logic, and role-based workflow rather than infrastructure.

9 Project scope and deliverables

...iteration-friendly

9.1 Delivered so far

... Weeks 1–2

- Week 1: finalized scope and tiering across 15 modules, full ERD, complete Postgres schema, backend/frontend skeleton with a live `/health` check.
- Week 2: JWT auth (register/login/me), Staff & Employee Management (records, shift scheduling with conflict rejection, unit tested), attendance clock-in/out, role-aware frontend dashboard.

9.2 Subsequent deliverables

- Ops Core (flights, gates, resources, turnaround) and delay handling with cascading recalculation.
- Booking portal, connecting-flight risk, baggage management, check-in.
- Emergency handling, security module, predictive analytics, support chatbot.
- Parking, facilities directory, accessibility & medical assistance.
- Admin dashboard rolling up all modules.

10 Risks and mitigations

- **Shared-core coupling risk:** a bug in the core schema affects every module; mitigated by finalizing the ERD and schema in Week 1, before feature work, and unit-testing shared services like conflict detection.
- **Scope size (15 modules):** mitigated by the explicit Tier 1/Tier 2 split, so depth is spent where it matters and the remainder is deliberately kept simple.
- **Real-time correctness:** cascading updates (delays, gate changes) are harder to get right than simple CRUD; mitigated by building this pattern early (Ops Core, Tier 1) and reusing the same conflict-service pattern established in Week 2.
- **Deployment drift:** mitigated by deploying the skeleton in Week 1 and shipping every subsequent module against that same live environment.

11 Summary

This project proposes a deployable, multi-module Airport Management System built on one shared operational data layer, demonstrating that a well-designed core prevents the data duplication and cascade-delay problems that fragmented airport systems typically suffer from. It meets the course criteria by shipping a deployed skeleton in Week 1, tiering module complexity deliberately, reusing a single unit-tested conflict-detection pattern across multiple modules, and defining cross-module consistency as the primary, attributable evaluation metric.